API Reference

This documentation outlines the two-step process for extracting structured JSON data from voice call recordings. The pipeline fetches the raw audio from Ultravox and processes it through a self-hosted LLM (Llama 3.2) using a custom data extraction schema, delivering the final data to your webhook.

Step 1: Fetch Call Recording

Retrieve the raw `.wav` audio file for a specific call session.

Endpoint: `GET https://api.ultravox.ai/api/calls/{call_id}/recording`

Request Headers

Key	Value	Description
`X-API-Key`	`string`	**Required.** Your Ultravox API key.

Path Parameters

Parameter	Type	Description
`call_id`	`string (uuid)`	**Required.** The unique identifier for the Ultravox call session.

Response

- **`200 OK`**: Returns the raw `audio/wav` binary file.

Step 2: Process Audio & Extract Data

Send the fetched audio file alongside your extraction instructions to the AI processing server. This endpoint runs asynchronously; it will immediately return a queued status and POST the final results to your designated webhook.

Endpoint: `POST http://72.61.206.216:8000/process-call`

Content-Type: `multipart/form-data`

Form Data Parameters

Parameter	Type	Description
`file`	`file`	**Required.** The `.wav` audio file retrieved from Step 1.
`webhook_url`	`string`	**Required.** The destination URL where the final JSON payload will be sent.
`uid`	`string`	**Optional.** The user ID or client identifier. Passed blindly to the webhook for routing.
`call_id`	`string`	**Optional.** The CRM or internal call ID. Passed blindly to the webhook.
`system_prompt`	`string`	**Required.** The core instructions for the AI. *Example: "You are a data extractor. Read the transcript and extract the details. Do not output anything else."*

required_fields	string (JSON)	**Required.** A JSON object defining the exact keys you want extracted, with values containing "toddler-proof" instructions for the AI to find that data.

Example curl Request

Bash

```bash
curl -X POST http://72.61.206.216:8000/process-call \
  -F "file=@/path/to/downloaded_audio.wav" \
  -F "webhook_url=https://your-domain.com/webhook/receive-data" \
  -F "uid=usr_998877" \
  -F "call_id=call_abc123" \
  -F "system_prompt=You are a data extractor. Read the transcript and extract the details. Do not output anything else." \
  -F "required_fields={\"customer_name\": \"Look for 'Am I speaking to [Name]'. Put that name here.\", \"agent_name\": \"Look for 'My name is [Name] calling from'. Put that name here.\", \"primary_interest\": \"The car model they want to buy\"}"
```

Immediate Response (Synchronous)

JSON

```json
{
  "status": "queued",
  "call_id": "call_abc123",
  "message": "Audio received. Processing in the background."
}
```

Step 3: Webhook Payload Delivery

Within 1–3 minutes, the processing server will POST the complete transcript and structured data to your `webhook_url`.

Method: `POST`

Content-Type: `application/json`

Example Success Payload

JSON


```json
{
  "status": "success",
  "uid": "usr_998877",
  "call_id": "call_abc123",
  "raw_transcript": "Hello. Hello. Good morning. Am I speaking to Mujini Amos? ...",
  "extracted_data": {
    "customer_name": "Mujini Amos",
    "agent_name": "Fiona",
    "primary_interest": "Toyota RAV4"
  }
}
```


Example Error Payload

If the process fails (e.g., an invalid file format is uploaded), the webhook will still fire to notify your systems.

JSON


```json
{
  "status": "error",
  "uid": "usr_998877",
  "call_id": "call_abc123",
  "message": "[Errno 1094995529] Invalid data found when processing input: '/tmp/tmpi02l3yoo.wav'"
}
```